Beyond Static LLM Policies: Imitation-Enhanced Reinforcement Learning for Recommendation

Yi Zhang[1,2,*], Lili Xie[*], Ruihong Qiu[1], Jiajun Liu[2,1], Sen Wang[1]
[1]School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Australia
[2]Data61, CSIRO, Brisbane, Australia
uqyzha91@uq.edu.au, xaiverlilix@gmail.com, r.qiu@uq.edu.au, jiajun.liu@csiro.au, sen.wang@uq.edu.au

***Abstract*—Recommender systems (RecSys) have become critical tools for enhancing user engagement by delivering personalized content across diverse digital platforms. Recent advancements in large language models (LLMs) demonstrate significant potential for improving RecSys, primarily due to their exceptional generalization capabilities and sophisticated contextual understanding, which facilitate the generation of flexible and interpretable recommendations. However, the direct deployment of LLMs as primary recommendation policies presents notable challenges, including persistent latency issues stemming from frequent API calls and inherent model limitations such as hallucinations and biases. To address these issues, this paper proposes a novel offline reinforcement learning (RL) framework that leverages imitation learning from LLM-generated trajectories. Specifically, inverse reinforcement learning is employed to extract robust reward models from LLM demonstrations. This approach negates the need for LLM fine-tuning, thereby substantially reducing computational overhead. Simultaneously, the RL policy is guided by the cumulative rewards derived from these demonstrations, effectively transferring the semantic insights captured by the LLM. Comprehensive experiments conducted on two benchmark datasets validate the effectiveness of the proposed method, demonstrating superior performance when compared against state-of-the-art RL-based and in-context learning baselines. The code can be found at https://github.com/ArronDZhang/IL-Rec.**

***Index Terms*—Inverse Reinforcement Learning, Recommender Systems, Large Language Models**

I. Introduction

Recommender systems (RecSys) have become an essential component of digital platforms, reshaping user experiences in domains such as e-commerce, social media, and online entertainment [25], [32]–[35]. By analyzing user behaviors and preferences, RecSys can deliver highly personalized content, thereby enhancing user engagement and retention. Traditional RecSys, ranging from collaborative filtering [27] to deep learning-based approaches [22], [40], have significantly improved personalization capabilities. However, these methods often struggle with adapting to rapidly evolving user interests and optimizing recommendations for long-term user satisfaction, thereby necessitating more advanced solutions [6].

Recent advancements in large language models (LLMs) [1], [15], [43] have opened new opportunities for improving RecSys with superior generalization and contextual understanding abilities [48]. LLM-based recommenders can process diverse data modalities, including text, images, and structured data, to generate more informed and flexible recommendations. Additionally, LLMs enable interpretability by generating natural language explanations for recommendations with transparency and user-friendliness. Despite these advantages, integrating LLMs into RecSys still poses significant challenges, such as high computational costs, reliance on external API calls, and the difficulty of adapting static LLM-generated policies to dynamic user preferences [52].

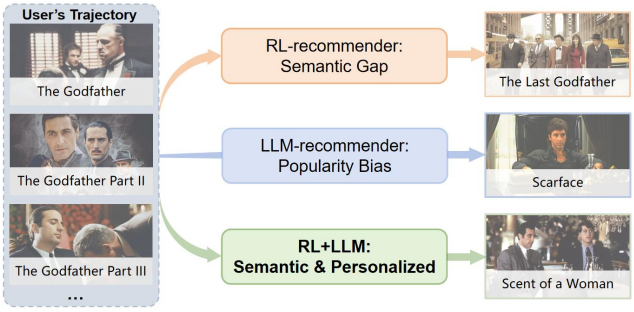


Fig. 1. Comparison of three recommendation strategies. RL4RS models often struggle to interpret user intent from textual input, resulting in less accurate suggestions. Static LLM policies tend to favor prevalent items of the same type, which may reduce recommendation diversity. The integration of RL into LLM recommendation exploits language understanding with adaptive and user-specific decision-making.

While most existing LLM-based methods utilize LLMs directly as recommendation policies [38], [51], two primary challenges arise in practice: (1) Online recommendation demands low-latency responses [8], [48], yet relying solely on LLMs necessitates frequent calling of the API, resulting in **consistently** high latency; (2) outputs are prone to spurious associations and reinforcement of recurrent patterns, undermining their reliability as standalone recommendation policies [10], [20]. Conversely, reinforcement learning (RL)-based sequential decision approaches have been adopted to optimize long-term engagement, but they often rely on coarse feedback signals and fail to capture nuanced semantic dependencies in user histories. As illustrated in the movie recommendation example in Fig. 1, when a user's interactions center on *The Godfather Trilogy*, standard reinforcement learning for recommender system (RL4RS) methods may replicate substring matches (e.g., "Godfather") without recognizing deeper context such as recurring cast members. Recent efforts [3],

[*] indicates equal contribution.

2025 IEEE International Conference on Data Mining (ICDM) | 979-8-3315-9599-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDM65498.2025.00098

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

[10], [38] seek to mitigate these issues by integrating the generative strengths of LLMs with the sequential decision-making capabilities of reinforcement learning. Nevertheless, methods involving LLM fine-tuning [3], [10] incur substantial computational overhead and are time-intensive. Meanwhile, approaches that incorporate historical feedback into subsequent trajectories through prompting [38] partially mimic gradient-based learning but still suffer from persistent latency and produce **sub-optimal** recommendation policies.

To address these challenges, an RL4RS approach, **IL-Rec**: Imitation-Enhanced Reinforcement Learning for Recommendation, is proposed in which LLM-generated demonstrations are imitated to eliminate continual online LLM invocation and mitigate inherent hallucinations and biases. Specifically, in-context LLM trajectories are employed as demonstrations, removing the need for LLM fine-tuning and reducing computational overhead. Reward models corresponding to the LLM policy are extracted using inverse reinforcement learning. Simultaneously, an RL policy is trained, guided by cumulative rewards of the LLM-generated trajectories, which are used as importance weights. In this way, the semantic information captured by the LLM is leveraged, the RL training process is accelerated, and the influence of hallucinations and biases is alleviated. The proposed method is validated on two benchmark datasets and shown to outperform state-of-the-art RL4RS and in-context learning baselines. The key contributions of this paper are summarized as follows:

- Limitations of LLM-based in-context learning for RecSys are identified, and reinforcement learning is leveraged to establish a low-latency, semantically informed recommendation policy framework.
- An inverse reinforcement learning method is utilized to extract reward models from LLM demonstrations, accelerating RL policy training and enabling superior performance beyond the original LLM demonstrations.
- Extensive experiments on two benchmark datasets are conducted, demonstrating the proposed method consistently outperforms state-of-the-art RL4RS and strong static LLM baselines.

II. Related Work

A. Reinforcement Learning for RecSys

Owing to the sequential decision nature of RecSys, a recommendation task can be formulated as a Markov Decision Process (MDP). In the formulation of RL, some works try to figure out a practical way to encode relevant information into the state representation [19], [22]. Some focus on adapting existing RL methods to tackle special challenges in RecSys, such as huge action space [4] and data sparsity [5]. As training RL policies online is expensive, offline RL appears to be a feasible solution [24], [55]. CIRS [14] introduces causal inference to capture the user preference and mitigate the filter bubble problem. DORL [11] proposes an effective entropy penalty to encourage offline exploration during recommendation policy learning, alleviating the Mathew effects. ROLeR [54] identifies the inaccuracy of offline reward function estimation, exploiting a non-parametric reward shaping method to refine the world models. Though these methods show promising performance, the abundant text information in offline data is seldom utilized.

B. Large Language Models for RecSys

Diving into the practical scenarios of RecSys, such as movie platforms [17], short-video platforms [12], [13], and music platforms [2], rich text can be found, including the genres, introduction of the movies or music. This information provides the potential to deal with the typical challenges in RecSys, such as cold-start problems [37], [48]. LLMs can process vast amounts of textual data [1], [15], [43], capturing nuanced user preferences and providing contextually relevant suggestions. PrefRec [50] proposes to learn reward functions from offline trajectories with RLHF [41]. RLMRec [36] utilizes LLMs to learn informative representations to facilitate recommendations. FaiRLLM [53] focuses on the fairness issue in LLM for RecSys. Though RecSys benefits from embracing LLMs, challenges are also introduced, including high computational costs and the need for effective fine-tuning to align with specific recommendation tasks.

C. Imitation Learning for RecSys

While reinforcement learning (RL) has been widely adopted in sequential recommendation to optimize long-term user engagement, it often suffers from reward misalignment, where the designed reward signals fail to reflect users' true preferences. To address this, imitation learning (IL) has emerged as a promising alternative by learning user policies directly from expert behavior. Behavioral Cloning (BC), a simple yet effective IL method, has been applied in recommender systems to mimic user actions from historical trajectories. For example, Stratified Expert Cloning (SEC) [26] introduces expert stratification to enhance long-term retention. Advanced IL methods, such as Inverse Reinforcement Learning (IRL) [30], which infers reward functions behind expert behaviors and Adversarial Imitation Learning (AIL) [18], which leverages generative adversarial networks to directly learn the policy and reward function simultaneously, extends IL's abilities. InvRec [7] combines IRL and AIL via a Generative Adversarial Imitation Learning (GAIL)-based framework, while SHIL [46] decomposes complex tasks using hierarchical imitation.

Recent studies adopt imitation as a knowledge transfer paradigm to implicitly capture user intent in offline settings. DLLM2Rec [8] distills knowledge from LLMs to lightweight recommenders, and LEA [45] treats LLMs as simulators to support RL training. Unlike prior work relying on optimal demonstrations, the proposed method leverages LLMs to generate diverse, sub-optimal trajectories and applies GAIL to learn robust policies from imperfect data, reducing dependence on expert supervision and lowering inference costs.

III. Preliminaries

A. Model-based Offline RL

This part details the concepts of model-based offline reinforcement learning (RL). An RL task can be formulated as

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

a Markov Decision Process (MDP) and described by a 5-element tuple $\mathbf{G} =< \mathbf{S}, \mathbf{A}, T, r, \gamma >$ [42]. Within the MDP, $\mathbf{S}$ stands for the state space, which is the set of all possible states. Similarly, $\mathbf{A}$ is the action space defining all possible actions. $\mathbf{s} \in \mathbf{S}$ or $\mathbf{a} \in \mathbf{A}$ denote one state or action of the current MDP. T is the transition function describing the dynamics of the environment. In deterministic settings such as in RecSys, $\mathbf{s_{t+1}} = T(\mathbf{s_t}, \mathbf{a_t})$ offers the next state given current state and action. r is the reward function, $r(\mathbf{s_t}, \mathbf{a_t})$, which stands for the instant reward after conducting action $\mathbf{a_t}$ in state $\mathbf{s_t}$. γ is the discount factor balancing the current reward with future rewards. The goal of a MDP is to learn a policy π which can maximize the cumulative reward, *i.e.*, $G_t = \sum_{\mathbf{s=s_0}}^{s_T} \gamma^t r(\mathbf{s_t}, \mathbf{a_t})$, where $\mathbf{s_0}$ and $\mathbf{s_T}$ are the initial and terminal states. The learning of policies usually requires quantities of interactions between the policies and the environments. In some scenarios like RecSys, online interactions are expensive while offline logs are easily available.

Leveraging offline data to mitigate the reliance on costly online interactions forms the foundation of offline RL, where the data consists of interactions, *i.e.* $D = \{(\mathbf{s_t}, \mathbf{a_t}, \mathbf{r_t}, \mathbf{s_{t+1}})\}_{t=0}^{T}$, between the environment and behavior policies. Model-based offline RL methods simulate the online environments with offline data, *i.e.*, D, by explicitly modelling the transition functions, *i.e.*, $\hat{T}$, or reward functions, *i.e.*, $\hat{r}$. These simulated environments are called world models. The policies are learned by interacting with the world models as online policy training.

B. Inverse RL

Adversarial imitation learning (AIL) [18] is employed in this work, where a discriminator, *i.e.,* D, is utilized to differentiate the learning policy from the expert policy. And the objective of the learning policy is to "deceive" the discriminator that its trajectories are the expert demonstrations. Thus, the reward of the learning policy comes from the incorrectness of the discriminator, and that of the discriminator comes from the classification correctness. The objective of the two agents is:

$$\begin{aligned} \min_{\theta} \max_{\psi} & \mathbb{E}_{(s,a)\sim\rho_{\pi_\theta}}[logD_\psi(s,a)] \\ & + \mathbb{E}_{(s,a)\sim\rho_{\pi_E}}[log(1-D_\psi(s,a))], \end{aligned} \tag{1}$$

where θ and ψ denote the parameters of the learning policy and the discriminator, respectively. ρ denotes the corresponding distribution of the policy.

C. Problem Formulation

As the recommendation process can be regarded as a sequential decision task, it can be formulated by an MDP and tackled with RL methods. In RecSys, an action $\mathbf{a} \in \mathbf{A}$ represents recommending a specific item, and a state $\mathbf{s} \in \mathbf{S}$ details the conditions before recommending this item. The modelling of states in RL for RecSys is widely studied [19]. In this paper, a common way is adopted which takes the recently interacted items and side information of users, such as gender and age into consideration. The transition function of RecSys is usually handled by a sequence model, for instance, GRU [22] and Transformer [44], which encodes the item information into $\mathbf{s_t}$ to obtain $\mathbf{s_{t+1}}$. The reward functions of RecSys are often modelled by the immediate feedback after recommending an item to a user. They can be the ratings of the users towards the watched movies in movie platforms or the stars of the videos in short-video platforms. The goal of RecSys is to maximize the long-term user satisfactory and user engagement, which is captured by the cumulative rewards.

IV. Method

The proposed method can learn from offline expert demonstrations obtained by frozen LLM policies, which eases the need for persistent calling of LLM APIs. The collection of LLM expert demonstrations will be illustrated in the first subsection. Then, the inverse RL process is introduced to learn from the above demonstrations.

A. LLM Demonstrations

The demonstrations are generated by LLM expert, including four modules: Reflector, Planner, LLM Actor, and LLM Critic. Prompts examples can be found in Appendix A.

1) ***Reflector****:* It processes completed interaction episodes to generate textual reflections, facilitating planning and decision-making. Given an episode $\mathcal{E} = \{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t)\}_{t=1}^{T_k}$, the reflector generates a reflection $\mathbf{m}_k$ by a summarization function f_{ref} implemented with LLM:

$$\mathbf{m}_k = f_{\text{ref}}(\mathcal{E}, \mathcal{I}_r). \tag{2}$$

$\mathcal{I}_r$ is the instructions for the reflector with few-shot examples. The template of reflection is provided in Fig. 4. The generated reflection $\mathbf{m}_k$ is stored in the memory module of the Planner, *i.e.*, $\mathcal{M}_P$, and retrieved for guiding decision-making:

$$\mathcal{M}_P \leftarrow \mathcal{M}_P \cup \{\mathbf{m}_k\}. \tag{3}$$

The reflector is responsible for summarizing macro-level information throughout the recommendation process.

2) ***Planner****:* It generates high-level textual plans $\mathbf{g}_t$ at each time step. These plans capture users' preferences and facilitate the LLM actor to execute actions. Given the user state $\mathbf{s}_t$, which is the textual description of the past interactions, *i.e.*, $\mathcal{E}_t$, the planner retrieves relevant historical reflections $\mathbf{M}_s$ from the planner's memory module using Facebook AI Similarity Search (FAISS)-based [21] similarity search:

$$\mathcal{M}_p = \{m' \in \mathcal{M}_P | \text{sim}(m', s_t) > \tau_P\}, \tag{4}$$

where $\text{sim}(\cdot,\cdot)$ is the similarity function measuring the alignment between the current state and stored reflections. τ_P is the similarity threshold of reflections, after Min-Max normalization to $[0, 1]$. Subsequently, the Planner generates textual guidance $\mathbf{g}_t$ via an LLM-based planning function:

$$\mathbf{g}_t = f_{\text{plan}}(\mathbf{s}_t, \mathcal{M}_p, \mathcal{I}_P). \tag{5}$$

$\mathcal{I}_P$ denotes instructions of the Planner, including few-shot examples. Fig. 5 illustrates the structure of the prompt template. The Planner provides direct guidance for the item recommendation, enabling closer alignment with user preferences.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

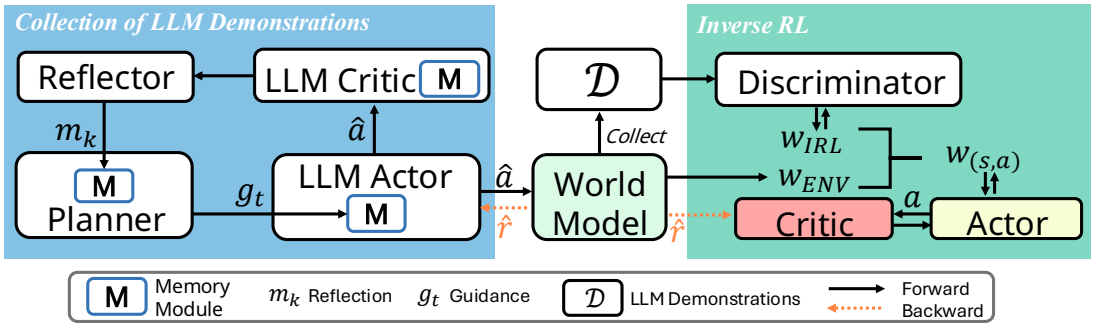


Fig. 2. The left part illustrates the collection of demonstrations from the LLM policy. Expert demonstrations are synthesized through the collaboration of the Planner, Reflector, LLM-Actor, and LLM-Critic. Subsequently, as shown on the right, adversarial inverse reinforcement learning (IRL) is employed to extract an effective reward function by training a discriminator to distinguish expert-generated trajectories from those produced by the current policy interacting with the learned world model. Through iterative interactions with the world model, the policy is progressively refined to surpass the original LLM expert demonstrations, while mitigating overfitting to artifacts of the simulated environment.

*3) **LLM Actor**:* It generates executable actions at each step t through a two-stage process: action generation and candidate selection. The inputs of LLM Actor include:

(I) **Guidance:** The guidance text $\mathbf{g}_t$ from the Planner.

(II) **Actor Memory Retrieval:** The actor retrieves relevant information from its memory module using FAISS-based similarity search:

$$\mathcal{M}_a = \{m' \in \mathcal{M}_A | \text{sim}(m', \mathbf{s}_t) > \tau_A\}, \tag{6}$$

where τ_A is the similarity threshold after normalization.

(III) **Instructions for LLM Actor:** $\mathcal{I}_A$ consists of a request and a few examples. The Actor function, *i.e.*, f_{act}, is also implemented as a frozen LLM:

$$\hat{\mathbf{a}}_t = f_{\text{act}}(\mathbf{s}_t, \mathbf{g}_t, \mathcal{M}_a, \mathcal{I}_A). \tag{7}$$

For ease of understanding, an illustrative prompt template of LLM-actor is provided in Fig. 6.

Since the generated $\hat{\mathbf{a}}_t$ may not exactly coincide with any item embedding, it is regarded as an indicator of the candidate set, and the item with the highest cosine similarity is chosen as the final action:

$$a_t = \arg\max_{\mathcal{I}} \sum_{i \in \mathcal{I}} \text{sim}(\hat{\mathbf{a}}_t, \mathbf{e}_i), \tag{8}$$

where $\mathcal{I}$ is item set and $\mathbf{e}_i$ stands for item embedding.

After each recommendation, the Actor updates its memory with the new state-action pair for future retrieval.

*4) **LLM Critic**:* It estimates the value function to evaluate the actor's action, and the prompt template is shown in Fig. 7. It first retrieves relevant historical state-value pairs from its memory module:

$$\mathcal{M}_c = \{m' \in \mathcal{M}_C | \text{sim}(m', \mathbf{s}_t) > \tau_C\}, \tag{9}$$

where τ_C is the similarity threshold. The retrieved information, combined with the critic's instructions, *i.e.*, $\mathcal{I}_C$, is fed into the LLM-based critic to estimate the state value:

$$V_{\text{LLM}}(s_t) = f_{\text{cri}}(\mathbf{s}_t, \mathcal{M}_c, \mathcal{I}_C). \tag{10}$$

To stabilize training, the advantage function is introduced following Advantage Actor Critic (A2C) [28]:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t), \tag{11}$$

where $Q(s_t, a_t) = r_t + \gamma V_{\text{LLM}}(s_{t+1})$ is also obtained from the estimated value function of the LLM Critic. At the end of each step, the Critic updates the memory modules with the new state-value pairs.

Through the interaction between the world model and the LLM modules, the LLM expert generates expert demonstrations $\mathcal{D}_E$, structured as sequential state-action trajectories:

$$\mathcal{D}_E = \{\tau_u\}_{u=1}^{U}, \tau_u = \{(s_1, a_1), (s_2, a_2), \ldots, (s_{T_u}, a_{T_u})\}. \tag{12}$$

Each state-action pair is encoded into textual embeddings using the LLaMA [43] model, ensuring semantically coherent and expressive representations for subsequent AIRL, bridging the expert policy generation and imitation learning.

B. Adversarial Inverse RL

In order to extract a reward function from the LLM-generated demonstrations, an adversarial inverse reinforcement learning (IRL) procedure is employed. A discriminator D_ψ is introduced to distinguish between the trajectories generated by the current learning policy π_θ and the expert demonstrations $\mathcal{D}_E$. It is interpreted as the probability that (s, a) originates from the policy rather than from the expert.

During the update of the discriminator, trajectories τ_θ are sampled by rolling out π_θ in the learned world model $\hat{T}$. Simultaneously, batches of state–action pairs are drawn from $\mathcal{D}_E$. The discriminator parameters ψ are then optimized to minimize the cross-entropy loss, similar to Eq. 1:

$$\begin{aligned}\mathcal{L}_D(\psi) = &\mathbb{E}_{(s,a)\sim\rho_{\mathcal{D}_E}}\left[\log\left(1 - D_\psi(s,a)\right)\right] \\ &- \mathbb{E}_{(s,a)\sim\rho_{\pi_\theta}}\left[\log D_\psi(s,a)\right].\end{aligned} \tag{13}$$

Correspondingly, the reward function is defined as:

$$r_{\text{IRL}}(s,a) = -\log D_\psi(s,a). \tag{14}$$

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

Therefore, transitions that are more likely to be classified as "expert" yield a higher reward.

*1) **Demonstration Weighting**:* Since the LLM-generated demonstrations $\mathcal{D}_E$ are known to be suboptimal, a weighting scheme is introduced to emphasize high-quality transitions while de-emphasizing those that are likely to be suboptimal. First, for each demonstration transition (s, a), the advantage $A_{\text{demo}}(s, a)$ is estimated using the world model's reward function, *i.e.,* $\hat{r}$, and a value network V_{demo}. Specifically, given a demonstration trajectory $\tau = \{(s_t, a_t, r_t)\}_{t=1}^{T}$, the return,

$$G_t \;=\; \sum_{k=t}^{T} \gamma^{\,k-t}\, r_k, \tag{15}$$

is computed via rollouts in the world model. $V_{\text{demo}}(s_t)$ is then estimated by fitting a neural network to minimize the mean-squared error between the predicted value and the empirically observed return. The advantage is defined as:

$$A_{\text{demo}}(s_t, a_t) \;=\; Q_{\text{demo}}(s_t, a_t) \;-\; V_{\text{demo}}(s_t). \tag{16}$$

Next, an environment-based weight can be computed as:

$$w_{\text{env}}(s, a) \;=\; \exp\!\big(\tfrac{1}{\beta}\, A_{\text{demo}}(s, a)\big), \tag{17}$$

where β is a temperature hyperparameter. Transitions with positive advantage are thereby assigned larger weight.

In parallel, an IRL-based weight is defined by:

$$w_{\text{IRL}}(s, a) \;=\; \Big(\tfrac{1}{D_\psi(s,a)} - 1\Big)^{\gamma}, \tag{18}$$

where γ controls the confidence of the discriminator. Transitions with smaller $D_\psi(s, a)$ (i.e., the discriminator is confident that (s, a) resembles the expert) receive a higher w_{IRL}.

Finally, the two weights are combined into a single weight:

$$w(s, a) \;=\; \big[w_{\text{env}}(s, a)\big]^{\,\alpha} \;\times\; \big[w_{\text{IRL}}(s, a)\big]^{\,1-\alpha}, \tag{19}$$

with $\alpha \in [0, 1]$ controlling the trade-off. The combined weight $w(s, a)$ is normalized across the demonstration dataset and clipped to a predefined range to ensure numerical stability.

*2) **Policy Update**:* At each iteration, the policy π_θ is updated by combining two objectives:

(I) **Weighted Imitation Loss:** For demonstration transitions $(s, a) \in \mathcal{D}_E$, the imitation objective is defined as:

$$\mathcal{L}_{\text{imit}}(\theta) \;=\; -\,\mathbb{E}_{(s,a)\sim\mathcal{D}_E}\big[\, w(s, a)\, \log \pi_\theta(a \mid s)\big]. \tag{20}$$

This loss encourages the policy to imitate demonstration transitions in proportion to their weights.

(II) **Reinforcement Loss (Actor–Critic):** Let Q_ϕ and V_ϕ denote the critic networks for estimating the environment return under π_θ. The actor–critic loss is given by:

$$\begin{aligned}\mathcal{L}_{\text{RL}}(\theta) = -\,\mathbb{E}_{s\sim\mathcal{D}}\Big[&\mathbb{E}_{a\sim\pi_\theta}\big[Q_\phi(s, a)\big] \\ &- \alpha_{\text{ent}}\mathcal{H}\big(\pi_\theta(\cdot \mid s)\big)\Big],\end{aligned} \tag{21}$$

where $\mathcal{H}(\pi_\theta(\cdot \mid s))$ is the policy entropy and α_{ent} is an entropy temperature. Samples are drawn from a replay buffer $\mathcal{D}$ containing both environment-generated transitions and demonstration transitions (sampled with priority proportional to $w(s, a)$).

The total policy loss is defined as:

$$\mathcal{L}_{\text{policy}}(\theta) \;=\; \lambda_{\text{imit}}\, \mathcal{L}_{\text{imit}}(\theta) \;+\; \mathcal{L}_{\text{RL}}(\theta), \tag{22}$$

where λ_{imit} balances imitation and reinforcement learning. The critic parameters ϕ are updated using temporal-difference targets, while the discriminator D_ψ is periodically retrained with updated rollouts and demonstrations.

C. Overall Learning Pipeline

The complete training procedure, as detailed in Fig. 2 consists of three phases: (i) world model learning, (ii) demonstration processing and weighting, and (iii) policy optimization.

*1) **World Model Learning**:* Similar to existing model-based methods [11], [54], offline logs are utilized to learn the reward function, *i.e.*, $\hat{r}$, of the world model via supervised learning, such as DeepFM [16]. The transition function is handled by a state tracker, which is implemented as a sequential model, encoding historical interactions as the next state.

*2) **Demonstration Processing and Weighting**:* Then, the LLM expert demonstrations $\mathcal{D}_E$ are collected as described in Section IV-A. For each transition $(s, a) \in \mathcal{D}_E$:

(I) Advantage $A_{\text{demo}}(s, a)$ is computed via rollouts in the world model and a dedicated value network V_{demo}.
(II) The discriminator D_ψ is trained on $\mathcal{D}_E$ and on policy rollouts ρ_{π_θ} in the world model.
(III) Weights $w_{\text{env}}(s, a)$ and $w_{\text{IRL}}(s, a)$ are computed, and fused to obtain $w(s, a)$. These weights are then normalized and stored in a replay buffer $\mathcal{B}_{\text{demo}}$.

*3) **Policy Optimization**:* At last, the policy π_θ and critics (Q_ϕ, V_ϕ) are updated using both environment rollouts and weighted demonstration samples:

(I) A batch of transitions $\{(s_i, a_i, r_i, s_i')\}$ is sampled from a mixed replay buffer $\mathcal{B} = \mathcal{B}_{\text{env}} \cup \mathcal{B}_{\text{demo}}$, where $\mathcal{B}_{\text{env}}$ contains recent rollouts of π_θ, and $\mathcal{B}_{\text{demo}}$ contains demonstration transitions paired with weights $w(s, a)$.
(II) Critic networks are updated by minimizing the temporal-difference loss:

$$\mathcal{L}_{\text{critic}}(\phi) \;=\; \mathbb{E}_{(s,a,r,s')\sim\mathcal{B}}\Big[\big(Q_\phi(s, a) - y\big)^2\Big], \tag{23}$$

where the target $y = r + \gamma\, V_{\phi_{\text{target}}}(s')$, and ϕ_{target} denotes delayed parameters. Demonstration samples contribute to $\mathcal{L}_{\text{critic}}$ in proportion to their weights $w(s, a)$.
(III) The policy is updated by minimizing $\mathcal{L}_{\text{policy}}(\theta)$ in Eq. 22.
(IV) Periodically, the discriminator D_ψ is retrained using the updated demonstration weights and rollout data from π_θ.

The last two phases are repeated until convergence. By iteratively re-estimating demonstration weights and updating the policy via actor–critic and weighted imitation, the learned policy is driven to exceed the performance of the original LLM demonstrations, avoiding overfitting to simulator artifacts.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DATASET STATISTICS.

Dataset	Usage	# Users	# Items	Density
Steam	Train	6012	190365	0.145%
	Test	6012	190365	0.084%
Amazon	Train	3109	13864	0.788%
	Test	3109	13864	0.320%

V. EXPERIMENTS

In this section, extensive experiments are conducted to validate the effectiveness of the proposed method by answering the following research questions (RQs):

• (RQ1) How is the recommendation performance of IL-Rec compared to other baselines?

• (RQ2) How do the components and designs of IL-Rec contribute to its performance?

• (RQ3) How does the generalization capacity of LLMs influence the performance of IL-Rec?

• (RQ4) How is the robustness of IL-Rec against its critical hyperparameters?

A. Setup

*1) **Datasets**:* Experiments are conducted on two benchmark datasets following BiLLP [38] with statistics in Tab. I.

• **Steam** [22] contains user histories[1] from a game platform. The rating of a user toward a game is measured by the game time, ranging from 1 to 5, serving as the immediate rewards.

• **Amazon** [31] contains book rating histories[1]. The genres and titles of books serve as the text information. The range of ratings is also $[1, 5]$ and serves as an immediate rewards.

*2) **Baselines**:* The baseline methods can be classified into two types: offline reinforcement learning methods and LLM methods. For all model-based methods, similar to prior works [11], [14], [54], world models are trained through DeepFM [16], which means they share the same world models for fair comparison of their recommendation performance.

Offline RL:

• ***DQN* (2015)** [29] is a fundamental Q-learning-based method that employs deep neural networks to estimate the value function.

• ***A2C* (2016)** [28] is a fundamental actor-critic baseline which exploits the advantage function to stabilize training.

• ***SQN (2020)*** [49] employs a dual-head architecture, comprising a cross-entropy loss head and an RL-based head. The RL head is in charge of generating the final recommendations.

• ***CRR (2020)*** [47] is a model-free RL method that focuses on improving policy learning by explicitly avoiding out-of-distribution (OOD) actions, enhancing stability and robustness.

• ***BCQ (2019)*** [9] is an extension of deep Q-learning tailored for batch RL scenarios. It integrates a discrete-action mechanism that filters out uncertain data points, ensuring that policy updates rely only on high-confidence samples.

• ***CQL (2020)*** [23] is another model-free RL approach. It introduces a Q-value regularization mechanism within an actor-critic framework, aiming to constrain the learned policy and mitigate overestimation issues.

• ***DORL (2023)*** [11] is a model-based RL method which serves as a strong baseline on KuaiRec [12] and KuaiRand [13]. It reconstructs the uncertainty penalty and introduces an entropy penalty to ease the Mathew effect.

• ***ROLeR (2024)*** [54] is a non-parametric soft-label-based reward shaping method. It designs an uncertainty penalty tailored for model-based offline RL4RS.

LLM Methods:

• ***ActOnly*** serves as a naive LLM baseline which directly recommends items with LLM Actor-Critic.

• ***ReAct (2023)*** [51] integrates LLMs to simultaneously generate reasoning traces and execute task-specific actions in an interleaved fashion. By combining reasoning and decision-making within the same framework, it enhances the coherence between thought processes and actions.

• ***Reflextion (2023)*** [39] employs verbalized self-reflection based on task feedback, storing these reflections in an episodic memory buffer. The retained reflective insights improve decision-making in future interactions by allowing the model to refine its responses iteratively.

• ***BiLLP (2024)*** [38] is the current SOTA on Steam and Amazon-Book with respect to cumulative reward evaluation. It exploits the LLM Actor-Critic as a direct recommendation policy. The update of its policy relies on the update of the memory modules.

*3) **Evaluation and Metric**:* To systematically evaluate the effectiveness of IL-Rec, a structured evaluation protocol consistent with prior studies [11], [14], [38], [54] is retained. The evaluation focuses on key performance indicators that assess both short-term and long-term user satisfaction.

• R_{tra} measures the average cumulative reward over testing episodes. As it directly reflects long-term user engagement and satisfaction, it serves as the primary metric for evaluation.

• R_{each} denotes the mean single-step reward obtained during testing. This metric captures the immediate satisfaction level of users with each recommended item.

• Len represents the average interaction length across test trajectories, offering insights into the recommender system's ability to sustain user engagement.

The recommendation session is terminated under two conditions: (1) If, within the most recent N transitions, a user has received M items from the same category, the session ends. To ensure fair comparison with [38], $N = 50, M = 4$ for the Steam dataset and $M = 4, N = 15$ for Amazon is adopted, enforcing category diversity within every four interactions. (2) The session also ends when the interaction length reaches the predefined upper limit of 100.

*4) **Implementation**:* The choices of hyperparameters are summarized at here for readability. The temperature of the LLMs is set to 0.5 to ensure the flexibility of the generation. The temperature β in w_{env} is chosen from $\{0.1, 1.0, 10.0\}$. The trade-off coefficient α in weight fusion is turned in $\{0.25, 0.5, 0.75\}$. The entropy temperature α_{ent} is chosen from $\{0.01, 0.1, 0.2\}$. The imitation loss weight λ_{imit} is tuned

[1]https://github.com/jizhi-zhang/BiLLP

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE PERFORMANCE SUMMARY OF ALL METHODS ON STEAM AND AMAZON. (BOLD: BEST; UNDERLINE: RUNNER-UP)

Method	Steam			Amazon		
	Len	R_{each}	R_{traj}	Len	R_{each}	R_{traj}
SQN	2.183 ± 0.177	3.130 ± 0.050	6.837 ± 0.517	4.773 ± 0.059	4.303 ± 0.017	20.570 ± 0.245
CRR	4.407 ± 0.088	3.263 ± 0.427	14.377 ± 1.568	3.923 ± 0.162	4.537 ± 0.130	17.833 ± 1.129
BCQ	4.720 ± 0.343	3.997 ± 0.068	18.873 ± 1.092	4.847 ± 0.721	4.367 ± 0.053	21.150 ± 2.893
CQL	5.853 ± 0.232	3.743 ± 0.147	21.907 ± 0.299	2.280 ± 0.185	4.497 ± 0.039	10.263 ± 0.882
DQN	4.543 ± 0.693	4.500 ± 0.069	20.523 ± 3.618	4.647 ± 0.498	4.290 ± 0.083	19.923 ± 1.909
A2C	9.647 ± 0.848	4.367 ± 0.069	42.180 ± 3.937	7.873 ± 0.310	4.497 ± 0.026	35.437 ± 1.453
DORL	9.467 ± 0.862	4.033 ± 0.098	38.300 ± 4.173	7.507 ± 0.174	4.510 ± 0.014	33.887 ± 0.655
ROLeR	10.218 ± 0.783	4.208 ± 0.086	42.997 ± 3.448	8.604 ± 0.243	4.590 ± 0.046	39.492 ± 0.632
ActOnly	5.567 ± 0.160	4.537 ± 0.021	25.250 ± 0.637	6.383 ± 0.176	4.490 ± 0.008	28.660 ± 0.761
ReAct	11.630 ± 0.741	**4.559 ± 0.047**	42.990 ± 2.925	7.733 ± 0.450	4.603 ± 0.033	35.063 ± 1.106
Reflexion	12.690 ± 1.976	4.523 ± 0.026	57.423 ± 8.734	8.700 ± 0.535	**4.670 ± 0.073**	40.470 ± 2.954
BiLLP	15.367 ± 0.119	4.503 ± 0.069	69.193 ± 1.590	9.413 ± 0.190	4.507 ± 0.012	42.443 ± 0.817
IL-Rec (Ours)	**17.533 ± 2.602**	4.476 ± 0.324	**78.478 ± 3.267**	**11.136 ± 0.797**	4.330 ± 0.114	**48.219 ± 2.198**

over $\{0.1, 0.5, 2.0\}$. Adam is the default optimizer used for all the components of the RL Actor-Critic in inverse RL. The learning rate is set to 0.001. A single trial of IL-Rec requires about 12 GPU-hours on an NVIDIA Tesla A100 (40 GB HBM2), consisting of 3 GPU-hours for world model simulation, 3 GPU-hours for generating LLM demonstrations, and 6 GPU-hours for adversarial inverse RL. In comparison, BiLLP consumes 6 GPU-hours per trial. While IL-Rec roughly doubles the computational cost, it delivers substantial gains in long-term user satisfaction. Moreover, both the world model simulation and the demonstration generation are one-time costs, further illustrating the practicality of IL-Rec.

B. Overall Performance (RQ1)

To evaluate the effectiveness of IL-Rec, its performance is compared against a suite of offline RL and LLM-based baselines. Tab. II reports the results on the Steam and Amazon datasets.

Cumulative Reward (R_{traj}). IL-Rec is observed to attain the highest cumulative reward on both benchmarks. On Steam, IL-Rec achieves 78.478, surpassing the previous best, *i.e.,*, BiLLP, by 13.4%. On Amazon, it achieves 48.219, an improvement of 13.6% over BiLLP. These gains demonstrate that the proposed imitation-enhanced RL framework is able to learn policies that substantially improve long-term engagement.

Single-Step Reward (R_{each}). Competitive single-step rewards are produced by IL-Rec, 4.476 on Steam and 4.330 on Amazon, although it does not hold the top position. Methods based on direct LLM reasoning—ReAct and ActOnly —yield slightly higher immediate rewards, as their designs prioritize short-term engagement. However, their cumulative rewards remain considerably lower, underscoring the advantage of balancing immediate gains with long-term strategy via RL.

Interaction Length (Len)**.** The longest recommendation trajectories are produced by IL-Rec, indicating enhanced sustained engagement. On Steam, the average trajectory length of 17.533 represents a 14.1% increase over BiLLP's. On Amazon, the length of 11.136 corresponds to an 18.3% increase over the second best. This extension of interaction horizons is attributed to the integration of structured demonstration weighting and RL-driven policy refinement.

Baseline Analysis. Among the RL-based baselines, A2C, DORL, and ROLeR produce higher rewards and longer trajectories than value-based methods (DQN, BCQ, CQL), yet they remain inferior to IL-Rec, as they lack demonstration weighting and knowledge-guided imitation. Conversely, LLM-based methods such as BiLLP, ReAct, and Reflexion exhibit strong one-step reasoning but fail to optimize long-term returns due to their static policy updates. Overall, IL-Rec is shown to successfully bridge the gap between RL-based and LLM-based recommendation strategies, achieving superior long-term performance while retaining competitive immediate rewards.

C. Ablation Study (RQ2)

This section examines the contributions of each component within IL-Rec. Variants are defined as follows: "w/o w" sets $w(s, a) = 1$; "w/o w_{env}" uses only the IRL-based weight; "w/o w_{IRL}" uses only the environment-based weight; and "BiLLP" corresponds to disabling the entire adversarial IRL procedure. The results on Steam and Amazon are summarized in Tab. III.

Impact of Overall Weighting. When the weighting mechanism was removed (w/o w), cumulative reward (R_{traj}) is reduced 14.1% on Steam and 18.4% on Amazon. Interaction length (Len) likewise decreased by 14.5% and 22.5%, respectively. Single-step reward (R_{each}) remained effectively unchanged. These findings indicate that the weighting mechanism is essential for achieving long-horizon engagement.

Impact of Environment-Based Weight (w_{env}). When w_{env} is ablated, cumulative reward fell to 63.380 on Steam (–19.3%) and to 40.280 on Amazon (–16.5%), while Len dropped by 24.7% and 22.3%, respectively. In contrast, R_{each} increased by 7.2% and 7.6%, respectively. This indicates that environment-based advantage weighting is critical for optimizing long-term returns, at the cost of slight increases in immediate reward.

Impact of IRL-Based Weight (w_{IRL}). When w_{IRL} was removed, cumulative reward decreased to 71.350 on Steam (–9.1%) and to 44.000 on Amazon (–8.7%), with Len reductions of 10.6% and 1.0%. Single-step reward changed marginally (+1.7% on Steam, –7.9% on Amazon). These

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ABLATION STUDY ON IL-REC'S COMPONENTS AND DESIGNS.

Methods	Steam		
	Len	R_{each}	R_{traj}
BiLLP	15.367 ± 0.119	4.503 ± 0.069	69.193 ± 1.590
w/o w	14.990 ± 1.264	4.495 ± 0.664	67.380 ± 1.780
w/o w_{env}	13.210 ± 1.460	4.798 ± 0.394	63.380 ± 2.142
w/o w_{IRL}	15.670 ± 1.297	4.553 ± 0.535	71.350 ± 1.272
IL-Rec	17.533 ± 2.602	4.476 ± 0.324	78.478 ± 3.267
Methods	Amazon		
	Len	R_{each}	R_{traj}
BiLLP	9.413 ± 0.190	4.507 ± 0.012	42.443 ± 0.817
w/o w	8.630 ± 0.663	4.560 ± 0.056	39.350 ± 1.225
w/o w_{env}	8.650 ± 0.723	4.657 ± 0.399	40.280 ± 1.450
w/o w_{IRL}	11.030 ± 0.713	3.989 ± 0.058	44.000 ± 1.256
IL-Rec	11.136 ± 0.797	4.330 ± 0.114	48.219 ± 2.198

TABLE IV
IL-REC'S PERFORMANCE W/ DIFFERENT LLMS.

Methods	Steam		
	Len	R_{each}	R_{traj}
Llama2-7b	16.900 ± 1.296	4.246 ± 0.213	71.750 ± 1.611
Deepseek-R1-14b	17.600 ± 1.119	4.250 ± 0.134	74.800 ± 2.496
GPT3.5-16k	17.533 ± 2.602	4.476 ± 0.324	78.478 ± 3.267
GPT4-32k	27.100 ± 2.345	4.736 ± 0.286	128.346 ± 1.671
Methods	Amazon		
	Len	R_{each}	R_{traj}
Llama2-7b	9.820 ± 0.583	4.122 ± 0.593	40.480 ± 1.783
Deepseek-R1-14b	10.770 ± 0.688	4.160 ± 0.088	44.800 ± 1.241
GPT3.5-16k	11.136 ± 0.797	4.330 ± 0.114	48.219 ± 2.198
GPT4-32k	13.270 ± 0.552	4.832 ± 0.069	64.121 ± 1.064

results confirm that IRL-based weighting principally supports long-horizon performance, exerting only minor influence on immediate rewards.

Comparison with BiLLP. On Steam and Amazon, the recommendation policies learned by the BiLLP baseline are not as effective as IL-Rec in sustaining long-term user satisfaction. Interaction lengths and R_{each} values under BiLLP further highlight that IL-Rec's dual-weighting and RL integration are responsible for its superior long-term engagement and competitive single-step performance. In addition, the results also show that without the capacity of learning from sub-optimal demonstrations, purely imitating the LLM polices cannot learn effective recommendation policies.

D. Impact of LLM Generalization (RQ3)

To evaluate how the performance of IL-Rec varies with different large language models (LLMs), experiments are conducted with Llama2-7b-hf-chat, Deepseek-R1-Distill-Qwen-14b, GPT-3.5-turbo-16k, and GPT-4-32k. Tab. IV presents the results on the Steam and Amazon datasets.

Scaling LLMs Improves Performance. The results indicate that as the LLM's capability increases, IL-Rec consistently benefits from the enhanced reasoning and generalization abilities. Using GPT-4-32k leads to the highest cumulative reward (R_{tra}), achieving 128.346 on Steam and 64.121 on Amazon, which significantly outperforms the other variants. Additionally, the interaction length (Len) also sees a substantial increase, confirming that stronger LLMs contribute to prolonged engagement and better policy learning.

Performance with Smaller LLMs. Even when using smaller-scale LLMs such as Llama2-7b and Deepseek-R1-14b, IL-Rec achieves competitive results compared to BiLLP. Notably, IL-Rec with Llama-7b attains 71.75 in R_{tra} on Steam, which is comparable to BiLLP's performance. Similarly, the results of IL-Rec with Deepseek-R1-14b have already surpassed BiLLP's cumulative reward on Amazon. This suggests that IL-Rec can effectively leverage even moderately capable LLMs to achieve strong recommendation policies, reducing reliance on high-resource models.

Implications for Future Improvements. The significant performance gains observed with more powerful LLMs suggest that IL-Rec has room for further improvement as LLM architectures continue to advance. The increasing gap in R_{tra} between Llama2-7b and GPT-4-32k implies that better LLM generalization leads to better long-term decision-making. This trend reinforces the potential of future iterations of IL-Rec incorporating more sophisticated LLMs with improved retrieval augmentation and decision planning.

Policies learned from LLM demonstrations surpass the static LLM experts, confirming the value of combining demonstrations with reinforcement learning. Yet, the suboptimality of these demonstrations is only partially alleviated. The extent of improvement depends on factors such as the reliability of the weighting mechanism, demonstration coverage, and biases. These factors delineate the capability boundary of IL-Rec, and further extension will be investigated in future work.

E. Hyperparameter Sensitivity (RQ4)

In this part, the robustness of IL-Rec against four critical hyperparameters is evaluated: the temperature β in w_{env}, the trade-off coefficient α in weight fusion, the entropy temperature α_{ent} and the imitation loss weight λ_{imit}. The corresponding results can be found in Fig. 3.

Effect of β. (Fig. 3, top-left), the cumulative reward is seen to increase on Steam as β is raised, reaching its maximum at $\beta = 10$. On Amazon, the reward peaked at $\beta = 1$ before declining at higher values. This suggests that a larger temperature is beneficial for weighting environment advantage on Steam, while a moderate β is preferable for Amazon.

Effect of α. (Fig. 3, top-right). On Steam, the highest reward was obtained at $\alpha = 0.5$, whereas on Amazon the best performance occurred at $\alpha = 0.25$. Values above or below these points led to reduced long-term returns, indicating the importance of balancing environment and IRL weights.

Effect of α_{ent}. (Fig. 3, bottom-left), the entropy temperature α_{ent} achieved optimal cumulative reward at 0.1 for both benchmarks. Lower or higher entropy weights resulted in modest performance drops, confirming that moderate exploration is critical to stable policy improvement.

Effect of λ_{imit}. The imitation loss coefficient λ_{imit} was found to be most effective at dataset-specific values (Fig. 3, bottom-right): $\lambda_{\text{imit}} = 0.25$ on Steam and $\lambda_{\text{imit}} = 0.5$ on

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

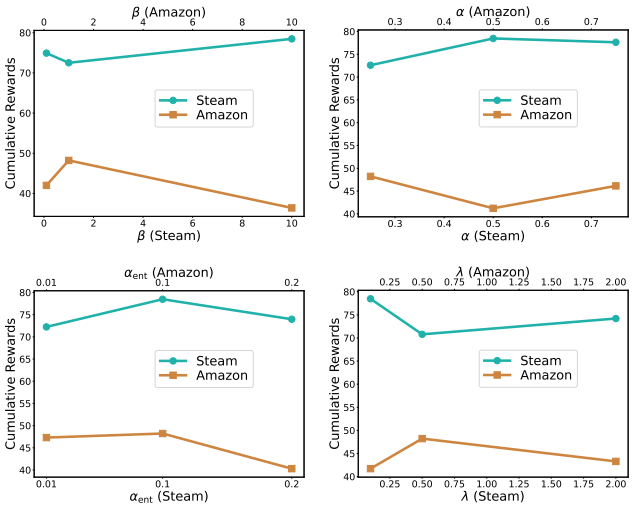


Fig. 3. Hyperparameter sensitivity of β, α, α_{ent}, and λ_{imit} on two datasets.

Amazon. Larger weights yielded diminishing returns or slight performance degradation.

Overall, IL-Rec's performance was shown to be relatively insensitive to variations in β, α, α_{ent}, and λ_{imit}. Although minor fluctuations in cumulative reward were observed, they remained confined to a narrow range, confirming the method's robustness across reasonable hyperparameter settings.

VI. Conclusion and Future Work

In this work, IL-Rec is proposed as a novel retrieval-enhanced hybrid RL framework that unifies imitation learning from LLM-generated demonstrations with independent RL policy refinement in a model-based offline recommendation paradigm. A discriminator-guided weighting mechanism is introduced, through which high-value transitions in demonstrations are selectively emphasized and the impact of sub-optimal ones is reduced, thereby enhancing recommendation policy learning. Empirical results on Steam and Amazon benchmarks demonstrated that IL-Rec consistently outperforms both state-of-the-art offline RL and LLM-based baselines in cumulative reward and interaction length.

Future directions include investigating more fine-grained aspects of LLM-generated demonstrations, such as their biases and coverage, to better understand their impact on policy learning. Extending IL-Rec to more diverse datasets across domains, real-world recommendation scenarios, and other sequential decision-making tasks is also part of the future work. In addition, adaptive strategies based on meta-learning or Bayesian optimization will be explored to automate hyperparameter tuning.

VII. Acknowledgments

This work is supported by projects DE200101610, DE250100919, CE200100025 funded by Australian Research Council, and CSIRO's Science Leader Project R-91559.

References

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv*, 2023.

[2] B. Brost, R. Mehrotra, and T. Jehan, "The music streaming sessions dataset," in *WWW*, 2019.

[3] H. Cai, Y. Li, W. Wang, F. Zhu, X. Shen, W. Li, and T.-S. Chua, "Large language models empowered personalized web agents," in *WWW*, 2025.

[4] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. H. Chi, "Top-k off-policy correction for a reinforce recommender system," in *WSDM*, 2019.

[5] M. Chen, B. Chang, C. Xu, and E. H. Chi, "User response models to improve a reinforce recommender system," in *WSDM*, 2021.

[6] X. Chen, S. Wang, J. McAuley, D. Jannach, and L. Yao, "On the opportunities and challenges of offline reinforcement learning for recommender systems," *TOIS*, 2023.

[7] X. Chen, L. Yao, A. Sun, X. Wang, X. Xu, and L. Zhu, "Generative inverse deep reinforcement learning for online recommendation," in *CIKM*, 2021.

[8] Y. Cui, F. Liu, P. Wang, B. Wang, H. Tang, Y. Wan, J. Wang, and J. Chen, "Distillation matters: empowering sequential recommenders to match the performance of large language models," in *RecSys*, 2024.

[9] S. Fujimoto, D. Meger, and D. Precup, "Off-policy deep reinforcement learning without exploration," in *ICML*, 2019.

[10] C. Gao, R. Chen, S. Yuan, K. Huang, Y. Yu, and X. He, "Sprec: Leveraging self-play to debias preference alignment for large language model-based recommendations," in *SIGIR*, 2025.

[11] C. Gao, K. Huang, J. Chen, Y. Zhang, B. Li, P. Jiang, S. Wang, Z. Zhang, and X. He, "Alleviating matthew effect of offline reinforcement learning in interactive recommendation," in *SIGIR*, 2023.

[12] C. Gao, S. Li, W. Lei, J. Chen, B. Li, P. Jiang, X. He, J. Mao, and T.-S. Chua, "Kuairec: A fully-observed dataset and insights for evaluating recommender systems," in *CIKM*, 2022.

[13] C. Gao, S. Li, Y. Zhang, J. Chen, B. Li, W. Lei, P. Jiang, and X. He, "Kuairand: An unbiased sequential recommendation dataset with randomly exposed videos," in *CIKM*, 2022.

[14] C. Gao, S. Wang, S. Li, J. Chen, X. He, W. Lei, B. Li, Y. Zhang, and P. Jiang, "Cirs: Bursting filter bubbles by counterfactual interactive recommender system," *TOIS*, 2023.

[15] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv*, 2025.

[16] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "Deepfm: a factorization-machine based neural network for ctr prediction," in *IJCAI*, 2017.

[17] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *TIIS*, 2015.

[18] J. Ho and S. Ermon, "Generative adversarial imitation learning," *NIPS*, 2016.

[19] J. Huang, H. Oosterhuis, B. Cetinkaya, T. Rood, and M. de Rijke, "State encoders in reinforcement learning for recommendation: A reproducibility study," in *SIGIR*, 2022.

[20] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *TOIS*, 2024.

[21] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE TBD*, 2019.

[22] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *ICDM*, 2018.

[23] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," in *NeurIPS*, 2020.

[24] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *CoRR*, vol. abs/2005.01643, 2020.

[25] Y. Li, T. Chen, Y. Luo, H. Yin, and Z. Huang, "Discovering collaborative signals for next poi recommendation with iterative seq2graph augmentation," in *IJCAI*, 2021.

[26] C. Lin, A. Xie, S. Liu, W. Wang, C. Wang, and Y. Liu, "Stratified expert cloning with adaptive selection for user retention in large-scale recommender systems," *arXiv*, 2025.

[27] B. M. Marlin and R. S. Zemel, "Collaborative prediction and ranking with non-random missing data," in *RecSys*, 2009.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.

[28] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *ICML*, 2016.
[29] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, 2015.
[30] A. Y. Ng and S. Russell, "Algorithms for inverse reinforcement learning," in *ICML*, 2000.
[31] J. Ni, J. Li, and J. McAuley, "Justifying recommendations using distantly-labeled reviews and fine-grained aspects," in *EMNLP*, 2019.
[32] R. Qiu, Z. Huang, and H. Yin, "Memory augmented multi-instance contrastive predictive coding for sequential recommendation," in *ICDM*, 2021.
[33] R. Qiu, Z. Huang, H. Yin, and Z. Wang, "Contrastive learning for representation degeneration problem in sequential recommendation," in *WSDM*, 2022.
[34] R. Qiu, J. Li, Z. Huang, and H. Yin, "Rethinking the item order in session-based recommendation with graph neural networks," in *CIKM*, 2019.
[35] R. Qiu, H. Yin, Z. Huang, and T. Chen, "Gag: Global attributed graph neural network for streaming session-based recommendation," in *SIGIR*, 2020.
[36] X. Ren, W. Wei, L. Xia, L. Su, S. Cheng, J. Wang, D. Yin, and C. Huang, "Representation learning with large language models for recommendation," in *WWW*, 2024.
[37] S. Sanner, K. Balog, F. Radlinski, B. Wedin, and L. Dixon, "Large language models are competitive near cold-start recommenders for language-and item-based preferences," in *RecSys*, 2023.
[38] W. Shi, X. He, Y. Zhang, C. Gao, X. Li, J. Zhang, Q. Wang, and F. Feng, "Large language models are learnable planners for long-term recommendation," in *SIGIR*, 2024.
[39] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *NIPS*, 2024.
[40] S. Sinha, A. Mandlekar, and A. Garg, "S4rl: Surprisingly simple self-supervision for offline reinforcement learning in robotics," in *CoRL*, 2022, pp. 907–917.
[41] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano, "Learning to summarize with human feedback," *NeurIPS*, 2020.
[42] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
[43] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *CoRR*, vol. abs/2307.09288, 2023.
[44] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.
[45] J. Wang, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Reinforcement learning-based recommender systems with large language models for state reward and action modeling," in *SIGIR*, 2024.
[46] L. Wang, R. Tang, X. He, and X. He, "Hierarchical imitation learning via subgoal representation learning for dynamic treatment recommendation," in *WSDM*, 2022.
[47] Z. Wang, A. Novikov, K. Zolna, J. S. Merel, J. T. Springenberg, S. E. Reed, B. Shahriari, N. Siegel, C. Gulcehre, N. Heess *et al.*, "Critic regularized regression," in *NeurIPS*, 2020.
[48] L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. Shen, C. Qin, C. Zhu, H. Zhu, Q. Liu *et al.*, "A survey on large language models for recommendation," *WWW*, 2024.
[49] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-supervised reinforcement learning for recommender systems," in *SIGIR*, 2020.
[50] W. Xue, Q. Cai, Z. Xue, S. Sun, S. Liu, D. Zheng, P. Jiang, K. Gai, and B. An, "Prefrec: Recommender systems with human preferences for reinforcing long-term user engagement," in *SIGKDD*, 2023.
[51] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *ICLR*, 2023.
[52] Z. Yue, S. Rabhi, G. d. S. P. Moreira, D. Wang, and E. Oldridge, "Llamarec: Two-stage recommendation using large language models for ranking," *arXiv*, 2023.
[53] J. Zhang, K. Bao, Y. Zhang, W. Wang, F. Feng, and X. He, "Is chatgpt fair for recommendation? evaluating fairness in large language model recommendation," in *RecSys*, 2023.
[54] Y. Zhang, R. Qiu, J. Liu, and S. Wang, "Roler: Effective reward shaping in offline reinforcement learning for recommender systems," in *CIKM*, 2024.
[55] Y. Zhang, R. Qiu, X. Xu, J. Liu, and S. Wang, "Darlr: Dual-agent offline reinforcement learning for recommender systems with dynamic reward," in *SIGIR*, 2025.

APPENDIX

A. Demonstration of Prompts

Prompt template to generate the reflection from Reflector	
Instruction:	You are an advanced reasoning agent that can improve based on self refection. You will be given a previous reasoning trial in which you were required to recommend items for the user. You were unsuccessful in retaining multiple interaction because user does not satisfied with the recommendation. In a few sentences, Diagnose a possible reason for failure and devise a new, concise, high level plan that aims to mitigate the same failure.
Input:	Here is the example: **{examples}**(END OF EXAMPLES). Here is the previous trial: **{previous reasoning trial}**(END OF TRIAL). Give me the Reflection directly.
Response:	The previous recommendation failed because I overlooked the user's explicit preference for suspenseful, tightly-paced films with twist endings, recommending slower-paced films instead. Next time, I should strictly adhere to user-stated preferences, ensuring recommendations closely match the user's specified criteria to maintain engagement.

Fig. 4. Prompt to generate the reflection from Reflector as in BiLLP [38]

Prompt template to generate the guidance from Planner	
Instruction:	Solve a recommendation task with interleaving Guidance, Action, Observation steps. Guidance can reason about the current situation and current user interest, and Action can be the following types:(1) recommend[item], which recommend an item to user based on user's interest. Your goal is to meet the user's interest as much as possible and make recommendations to users as many times as possible. You may take as many steps as necessary. Here are some examples， please strictly follow the example's templates to give guidance and recommend item for this user.
Input:	Here is the example: **{examples}**(END OF EXAMPLES). Here is the reflection: **{reflection}**(END OF REFLECTION). The user's viewing history is **{interacted items}**, please generate the guidance.
Response:	The user mainly prefers Role-playing games, but also enjoys Strategy and occasionally Puzzle games. I'll first suggest a popular Role-playing title, then introduce Strategy or Puzzle games to diversify the experience.

Fig. 5. Prompt to generate the guidance from Planner as in BiLLP [38]

Prompt template to generate the action from LLM-actor	
Instruction:	<Instruction of Planner>. Note that if the user is not satisfied with your recommendations, he will quit and not accept new recommendations. Here are some examples， please strictly follow the example's templates to recommend item for this user.
Input:	Here is the example: **{examples}** (END OF EXAMPLES). The guidance of the user is: **{guidance}** (END OF GUIDANCE). The retrieved information is: **{relevant memory}** (END OF RETRIEVAL). The user's viewing history is **{interacted items}**, please recommend item.
Response:	Recommend the movie "Memento".

Fig. 6. Prompt to generate the action from LLM-actor as in BiLLP [38]

Prompt template to generate the value from LLM-critic	
Instruction:	You are an expert in recommendation systems. You need to determine whether the recommended list can satisfy the user. Your answer must be a integer range from -5 to 5. (The higher the score, the higher the quality of the recommendation sequence, the more satisfying the user's interest, and the less annoying the user.)
Input:	Here is the example: **{examples}**(END OF EXAMPLES). The retrieved information is: **{relevant memory}** (END OF RETRIEVAL). The user's viewing history is **{interacted items}**， please generate the critic value.
Response:	Critic value: 5

Fig. 7. Prompt to generate the value from LLM-critic as in BiLLP [38]

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:30:29 UTC from IEEE Xplore. Restrictions apply.